Project Onboarding Guide

> Read this first. It tells you what every file is for, and the exact order to learn this project.

---

1. What Is This Project?

A production-grade e-commerce backend built as a **teaching platform**.

It uses the same tools and patterns as real production systems: Spring Boot microservices, Kafka event-driven architecture, Kong API Gateway, Kubernetes, and AWS-compatible deployment via Floci.

The goal is not to build a shopping app — it's to teach you how professional systems are designed, deployed, and operated.

---

2. The Repository Structure (What Every File Is For)

```
ecommerce/
│
├── README.md                    ← START HERE. Quick navigation + quick start.
├── docker-compose.yml           ← Runs the entire system locally with one command
├── pom.xml                      ← Parent Maven config — shared dependencies for all 6 servic
es
│
├── docs/                        ← All documentation (you are here)
│   ├── ONBOARDING.md            ← This file — read first
│   ├── ARCHITECTURE.md          ← System design diagrams + data model + security layers
│   ├── DEVELOPER_GUIDE.md       ← Day-to-day workflow: branch, PR, test, deploy
│   ├── DEPLOYMENT.md            ← How to deploy to Floci (simulated AWS)
│   ├── AUTOSCALING.md           ← How HPA (horizontal pod autoscaling) works
│   ├── MONITORING.md            ← Logs (Loki), metrics (Prometheus), dashboards (Grafana)
│   └── CI_CD.md                 ← GitHub Actions pipeline explained step by step
│
├── user-service/                ← One folder per microservice (same structure in all 6)
├── product-service/
├── order-service/
├── inventory-service/
├── payment-service/
└── notification-service/
│
├── kong/
│   └── kong.yml                 ← Kong API Gateway routing rules (which URL goes to which se
rvice)
│
├── k8s/                         ← Kubernetes manifests (deploy to EKS/Floci)
│   ├── namespace/namespace.yaml ← Creates 'ecommerce' and 'monitoring' namespaces
│   ├── postgres/postgres.yaml   ← 5× PostgreSQL StatefulSets (one per service)
│   ├── redis/redis.yaml         ← Redis StatefulSet
│   ├── kafka/kafka.yaml         ← Kafka StatefulSet (KRaft mode, no Zookeeper)
│   ├── kong/kong.yaml           ← Kong Deployment + ConfigMap with routing rules
│   ├── services/
│   │   ├── deployments.yaml     ← All 6 microservice Deployments + HPA
│   │   ├── configmap.yaml       ← Non-secret config (Kafka URL, Redis host, AWS endpoint)
│   │   └── secrets.example.yaml ← Template for secrets (passwords, JWT key) — never committe
d
│   └── monitoring/monitoring.yaml ← Prometheus, Grafana, Loki, Promtail
│
├── scripts/                     ← Bash scripts to set up AWS-equivalent infra via Floci
│   ├── .env                     ← Auto-written by scripts; stores IDs (VPC, ALB, etc.)
│   ├── 01-vpc.sh                ← VPC, subnets, IGW, NAT, route tables, security groups
│   ├── 02-alb.sh                ← ALB, WAF, ACM cert, Route53, listeners, target group
│   ├── 03-ecr.sh                ← ECR repos, build Docker images, push to Floci registry
│   ├── 04-eks.sh                ← EKS cluster, IAM roles, node groups, kubeconfig
│   ├── 05-deploy.sh             ← Apply all K8s manifests, register Kong in ALB
│   ├── 06-teardown.sh           ← Delete everything (EKS, ALB, VPC, ECR)
│   └── README.md                ← What each script does and AWS concepts covered
│
└── .github/
    ├── workflows/
    │   ├── ci-cd.yml            ← Main pipeline: test → build → scan → push → smoke test
```

```
    │   └── codeql.yml            ← Security scan (SAST) — runs on every PR
    └── dependabot.yml            ← Weekly automated dependency updates
```

---

3. Inside a Microservice (All 6 Are Identical in Structure)

Use `order-service/` as your reference:

```
order-service/
│
├── Dockerfile                                  ← How to build the Docker image
├── pom.xml                                     ← Maven dependencies for this service
│
└── src/
    ├── main/
    │   ├── java/com/ecommerce/orderservice/
    │   │   ├── OrderServiceApplication.java    ← Main class, starts Spring Boot
    │   │   ├── config/
    │   │   │   ├── SecurityConfig.java         ← JWT filter, which URLs need auth
    │   │   │   └── KafkaConfig.java            ← Kafka producer/consumer beans
    │   │   ├── controller/
    │   │   │   └── OrderController.java        ← REST endpoints (@GetMapping, @PostMappin
g)
    │   │   ├── service/
    │   │   │   └── OrderService.java           ← Business logic (create order, cancel orde
r)
    │   │   ├── repository/
    │   │   │   └── OrderRepository.java        ← Database queries (Spring Data JPA)
    │   │   ├── model/
    │   │   │   ├── Order.java                  ← @Entity — maps to 'orders' table
    │   │   │   └── OrderItem.java              ← @Entity — maps to 'order_items' table
    │   │   ├── dto/
    │   │   │   └── OrderRequest.java           ← What the API accepts (JSON body shape)
    │   │   ├── kafka/
    │   │   │   ├── OrderEventPublisher.java    ← Sends events to Kafka topics
    │   │   │   └── OrderEventConsumer.java     ← Receives events from Kafka
    │   │   └── exception/
    │   │       └── GlobalExceptionHandler.java ← Converts exceptions to HTTP responses
    │   │
    │   └── resources/
    │       ├── application.yml                 ← All config: DB URL, Kafka, Redis
    │       └── db/migration/
    │           └── V1__init.sql                ← Flyway: creates tables on first run
    │
    └── test/
        └── java/com/ecommerce/orderservice/
            └── OrderServiceTest.java           ← Unit tests (run in CI)
```

---

4. Recommended Learning Order

Follow this order. Each step builds on the previous one.

Step 1 — Run it first, understand it second

```
docker compose up -d
# Wait 2 minutes, then:
curl http://localhost:8000/api/products
curl -X POST http://localhost:8000/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"You","email":"you@test.com","password":"test1234"}'
```

Before reading code, see the system working. This gives you the "what" before the "how."

Step 2 — Read the Architecture doc

`docs/ARCHITECTURE.md` — Read the system diagram. Understand the flow:

> Client → Kong → Service → Kafka → Downstream service → DB

This is the mental model everything else builds on.

Step 3 — Understand Kong routing

`kong/kong.yml` — This file tells Kong: "When someone calls `/api/orders/*`, forward to `order-service:8083`."

```
routes:
  - name: orders-route
    paths: ["/api/orders"]
    strip_path: false       # ← CRITICAL: keep /api/orders in the URL when forwarding
services:
  - name: order-service
    url: http://order-service:8083
```

Without Kong, clients would need to know the port of every service. Kong is the single front door.

Step 4 — Trace one request through the code

Follow a `POST /api/orders` from start to finish:

1. `kong/kong.yml` → Kong routes to `order-service:8083`
2. `order-service/controller/OrderController.java` → `@PostMapping("/api/orders")`
3. `order-service/service/OrderService.java` → validates, saves to DB, publishes event
4. `order-service/kafka/OrderEventPublisher.java` → sends `order.created` event to Kafka
5. `inventory-service/kafka/InventoryEventConsumer.java` → receives `order.created`
6. `inventory-service/service/InventoryService.java` → reserves stock, publishes `inventory.updated`

7. `payment-service/kafka/PaymentEventConsumer.java` → receives `inventory.updated` (if success)
8. `payment-service/service/PaymentService.java` → processes payment, publishes `payment.processed`
9. `notification-service/kafka/NotificationConsumer.java` → receives, sends email via SES

This is the **Saga pattern** — each service does its job, then publishes an event for the next.

Step 5 — Read the database schemas

`*/src/main/resources/db/migration/V1__init.sql` in each service. These SQL files are the ground truth of what data each service owns.

```
-- order-service V1__init.sql
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  status VARCHAR(20),          -- PENDING, CONFIRMED, CANCELLED
  total_amount DECIMAL(10,2),
  created_at TIMESTAMP
);
```

Notice: **no user table in orderdb, no product table in orderdb.**

Each service only knows its own data. To know a user's name, order-service would call user-service's API (or receive it in the Kafka event payload).

Step 6 — Read one service end-to-end

Read `user-service/` completely. It's the simplest (no Kafka consumers):

- `model/User.java` → understand the entity
- `repository/UserRepository.java` → understand how DB queries work
- `service/UserService.java` → understand the business logic
- `controller/UserController.java` → understand the REST endpoints
- `config/SecurityConfig.java` → understand JWT authentication
- `application.yml` → understand configuration

Then apply this understanding to other services.

Step 7 — Understand the CI/CD pipeline

`docs/CI_CD.md` → Then read `.github/workflows/ci-cd.yml`. Understand what happens automatically when you open a PR and when you merge.

Step 8 — Deploy to Kubernetes via Floci

`docs/DEPLOYMENT.md` → Then run `scripts/01-vpc.sh` through `scripts/05-deploy.sh`. This teaches you AWS networking, EKS, ECR, ALB — with real CLI commands.

Step 9 — Explore monitoring

`docs/MONITORING.md` → Then run:

```
kubectl port-forward svc/grafana 3000:3000 -n monitoring
open http://localhost:3000
```

Explore logs (Loki) and metrics (Prometheus) from your running services.

---

5. Key Concepts — What You'll Learn Here

Concept	Where to See It	Doc
REST API design	`*/controller/*.java`	—
Spring Security + JWT	`user-service/config/SecurityConfig.java`	—
Event-driven architecture	`*/kafka/` in each service	ARCHITECTURE.md
Saga pattern	order → inventory → payment flow	ARCHITECTURE.md
Database-per-service	5 separate `postgres-*` in docker-compose	ARCHITECTURE.md
Redis caching	`product-service/service/ProductService.java`	—
Redis distributed locking	`inventory-service/service/InventoryService.java`	—
Flyway migrations	`*/resources/db/migration/V1__init.sql`	—
API Gateway pattern	`kong/kong.yml`	—
Docker multi-service	`docker-compose.yml`	—
Kubernetes deployments	`k8s/services/deployments.yaml`	DEPLOYMENT.md
HPA autoscaling	`k8s/services/deployments.yaml` (HPA blocks)	AUTOSCALING.md
CI/CD pipeline	`.github/workflows/ci-cd.yml`	CI_CD.md
VPC networking	`scripts/01-vpc.sh`	DEPLOYMENT.md
Load balancing (ALB)	`scripts/02-alb.sh`	DEPLOYMENT.md
Container registry (ECR)	`scripts/03-ecr.sh`	DEPLOYMENT.md
Managed Kubernetes (EKS)	`scripts/04-eks.sh`	DEPLOYMENT.md
Security scanning	`.github/workflows/ci-cd.yml` (Trivy, CodeQL)	CI_CD.md

Observability	`k8s/monitoring/monitoring.yaml`	MONITORING.md

---

6. Common Mistakes to Avoid

"Why is Kong returning 404?" Check `kong/kong.yml`. Every route must have `strip_path: false` or Kong will strip `/api/orders` from the URL before forwarding, and the service won't match any endpoint. **"Why is the order-service calling payment even when out of stock?"** Read `docs/ARCHITECTURE.md` → Kafka Event Flow. Inventory must publish `inventory.updated {success: true}` first. Payment only listens to that event, not `order.created`. **"Why does the service crash on startup?"**

1. Flyway can't connect to Postgres yet — look for `connect-retries: 15` in `application.yml`. Postgres takes 5-10s to start.
2. Kafka admin bean fails — `spring.kafka.admin.fail-fast: false` in `application.yml` prevents this.

"My changes aren't in the Docker image."

```
# After changing code, rebuild:
docker compose up -d --build user-service
```

"I pushed to main directly." Branch protection should prevent this. Always: feature branch → PR → develop → PR → main.

---

7. Your First Contribution — Step by Step

```
# 1. Clone and start
git clone https://github.com/SumedhDhamdhere/ecommerce-platform
cd ecommerce-platform
docker compose up -d
# Wait ~2 min for all services to start

# 2. Verify it works
curl http://localhost:8000/api/products

# 3. Create your feature branch
git checkout develop
git pull
git checkout -b feature/your-name-first-task

# 4. Make a small change (e.g., add a field to ProductController)
# Edit the file, save

# 5. Run tests for that service
cd product-service
mvn test

# 6. Test your change manually
curl http://localhost:8000/api/products

# 7. Push and open a PR
git add .
git commit -m "feat: add stock field to product response"
git push -u origin feature/your-name-first-task
# Open PR on GitHub → target: develop

# 8. Watch CI run (Tests + Coverage + SAST)
# If all green → request review from tech lead
# After approval → merge to develop → watch CI deploy automatically
```

---

8. Getting Help

- **Something broken?** Check `docker compose logs` first.
- **K8s pod crashing?** `kubectl describe pod -n ecommerce`
- **CI pipeline confused?** Read `.github/workflows/ci-cd.yml` and `docs/CI_CD.md`
- **Architecture question?** `docs/ARCHITECTURE.md`
- **Deployment question?** `docs/DEPLOYMENT.md` + `scripts/README.md`